Project ESCAPE: Does Entropy-Based Patching Find Syntax? Final Results Across Code and Natural Language

Priyanka Agarwal
2024101016

Shashwat Mukadam
2024102008

Divyansh Atri
2024113001

Abstract

We test whether the entropy-triggered patch boundaries of the Byte Latent Transformer (BLT) align with syntactic constituent starts in Python, C++ and English, beyond a density-matched null and beyond structure-blind segmenters. On a frozen held-out split of 13,040 files, start F1 exceeds the null in every language (margins +0.120, +0.129, +0.348; $p \leq 10^{-4}$), but against the strongest structure-blind baseline BLT underperforms it in all three languages: indentation and word-start rules segment better. Three further predictions (P3, P4 and the O5 residual) fail as stated, and follow-up tests explain each failure mechanically. The prose margin is largest only because F1 margins scale with target density; relative to the null, code aligns more strongly. The forced-opener reversal (P4) comes from scoring the wrong byte: the start precedes the keyword, while the bytes just after the keyword are segmented below chance. The memory-unsafe residual (O5) exists only after conditioning on identifier entropy, and without that conditioning it is negative. Beyond the proposal, a 14.3M-parameter student distilled from the 99.5M patcher keeps about 90% of the teacher's held-out margin, and the teacher's alignment peaks at a higher threshold ($\tau^* = 2$) without overtaking the baselines. Entropy is a genuine but shallow structural signal.

1 Introduction

Byte-level models remove a fixed subword vocabulary, but they must still decide where to spend computation. BLT groups bytes into variable-length patches whenever a small next-byte model's entropy exceeds a threshold τ (Pagnoni et al., 2025). If those boundaries fall where syntactic constituents open, the allocation tracks structure. But rare identifiers, word onsets and formatting also raise entropy, so proximity alone cannot separate these explanations.

ESCAPE asks five questions from our proposal. O1/P1: do boundaries align with constituent starts beyond a null that keeps each file's own patch lengths? P2: do they align more at starts than at ends? O2/P3: is code segmented differently from prose? P4: is alignment weaker for constructs whose opener is forced (`if`, `for`) than for open-ended ones? O3: does this differ between indentation-based Python and bracket-based C++? O4: does it survive code noise? O5: does extra compute near memory-unsafe C++ survive conditioning on identifier unpredictability? The mid-submission fixed the protocol and reported P1–P4 on the held-out split. This report completes O3–O5, explains the three failed predictions with tests designed for them, checks the statistics against clustering, parser-recovery and context artefacts, and, beyond the proposal, distils the patcher into a 7× smaller student and evaluates it under the same frozen protocol (§5).

Our contributions are: (i) a byte-exact extraction pipeline that restores the reference 512-token sliding window missing from the public conversion; (ii) a calibration-first protocol with a fixed-EOF density-matched null and structure-blind baselines as the burden of proof; (iii) held-out results in three languages with every number regenerated from retained per-byte artefacts; and (iv) mechanistic follow-ups showing that each failed prediction is a statement about which bytes carry entropy, not a failure of measurement.

2 Background and Related Work

BLT segments bytes dynamically so that expensive latent computation operates over patches (Pagnoni et al., 2025). We analyse its entropy patcher, not downstream accuracy. BPE fixes its merge table

in advance and spends the same units on boilerplate as on logic (Sennrich et al., 2016). ByT5 and CANINE remove the vocabulary but process every byte or character uniformly (Xue et al., 2022; Clark et al., 2022). BLT is the first model at scale whose segmentation is both byte-level and decided at inference time, so each boundary is a claim the model makes and can be checked against external structure. Structural probes recover syntax from trained representations (Hewitt and Manning, 2019; Feng et al., 2020), but they need a trained probe or code-specific objectives. We ask instead whether an unsupervised compression signal with no parser in the loop recovers constituents as a side effect. Statistical word segmentation has long placed boundaries by predictability (Brent, 1999; Goldwater et al., 2009), and it was always checked against observable word boundaries. We apply the same check to the patcher, with tree-sitter ASTs for code and benepar parses (Kitaev and Klein, 2018) for English. For the extension we use online knowledge distillation (Hinton et al., 2015).

3 Protocol

Data and checkpoint. The frozen manifest has 16,300 samples: 8,000 Python functions from CodeSearchNet (Husain et al., 2019), 8,000 C++ files from The Stack (Kocetkov et al., 2022) (the proposal named CodeSearchNet, which has no C++) and 300 WikiText-103 paragraphs (Merity et al., 2017), all restored and hash-verified. We load the 99.5M-parameter patcher of `itazap/blt-1b-hf` (revision `91aa6b8e`) in float32 rather than the proposal's bfloat16 (the bf16 weights upcast losslessly; on the 28 fidelity inputs 0.26% of boundaries differ between the two) with $\tau = 1.3354$ nats, no length cap, and the reference 512-token sliding window in 8,192-token chunks. Six of eight fidelity gates passed as declared; two criteria (tokenizer EOS, random-byte density) were amended, and the amended set passed on the cluster stack. The checkpoint's logits agree with an independent re-implementation to 1.05×10^{-4}, with identical boundaries. In October the corpus and teacher artefacts on the cluster were deleted and then regenerated. The regenerated run reproduced the frozen per-file, score and diagnostic files byte for byte.

Metric, null and baselines. Entropy at byte i is $H_i = -\sum_v p_\theta(v \mid x_{<i}) \log p_\theta(v \mid x_{<i})$ over 260 symbols. Predictions are interior entropy boundaries. No length cap is set, so every boundary after the forced first byte is entropy-triggered and none needs excluding as length-capped. Targets are the unique starts of non-wrapper constituents (outermost node wins ties), excluding byte 0 and file end: statement- and expression-level tree-sitter nodes in code and benepar phrases in English. Parser character offsets are round-tripped to UTF-8 byte offsets and validated before any scoring. Memory-unsafe regions for O5 are C++ statements containing pointer dereference or address-of (`*`, `&`), `new`, `delete`, unsafe C library calls, or unsafe casts. Maximum-cardinality one-to-one matching within k bytes gives pooled P, R and F1. Each language is split 20/80 by seeded SHA256 rank into calibration and test files, and only calibration files choose $k \in \{0, 1, 2, 4, 8\}$, by the median per-file F1 margin over the null; $k = 0$ was selected everywhere. The null shuffles each file's (length, reason) patch pairs except the last, which stays fixed so that file end is never a boundary, and so it preserves the patch-length multiset and boundary count. We use 10,000 seeded permutations per file and 2,000 paired file-bootstrap replicates. A structural reading must also beat boundaries at every newline, at indentation changes, and at every word start under the same targets and matching. No primary number changed after the mid-submission. Later test-split scorings are the pre-registered extension runs of §5, each scored once, and sensitivity or exploratory runs ($k = 1, 2$, the O4 rate sweep, HF full attention, the byte-balanced student), none of which selected a setting.

4 Results on the Proposal's Objectives

P1 and the structure-blind burden. Start F1 exceeds the null in every language (Table 1), with ratios of 4–5× in code. Against the strongest structure-blind baseline, however, BLT underperforms it in all three languages (Table 2): Python -0.242 [-0.247, -0.237] against indentation, C++ -0.173 against word starts, and English -0.274 against word starts. The margin is positive across languages, but its magnitude depends on target density. Most of what it tracks is still line and word segmentation.

Precision versus recall. The proposal's mechanism predicted precision above recall: cheap interiors and forced closing delimiters would rarely trigger, so boundaries would be few and well placed. Instead precision is below recall in all three lan-

Language	k	Test files	P	R	F1	H_0 F1	ΔF1 [95% CI]	p
Python	0	6,400	0.083	0.822	0.152	0.032	+0.120 [0.117, 0.122]	≤1e-4
C++	0	6,400	0.095	0.646	0.165	0.036	+0.129 [0.125, 0.134]	≤1e-4
English	0	240	0.342	0.894	0.495	0.147	+0.348 [0.341, 0.354]	≤1e-4

Table 1: Held-out entropy-boundary alignment with constituent starts (P1). k was frozen on the calibration split. H_0 F1 is the permutation mean of the fixed-EOF density-matched null (10,000 permutations); intervals are paired file bootstraps; p is the one-sided permutation value.

Lang.	BLT	NL	Indent	Word	BLT−best [95% CI]
Python	0.152	0.000	0.394	0.245	-0.242 [-0.247, -0.237]
C++	0.165	0.041	0.281	0.339	-0.173 [-0.181, -0.167]
English	0.495	0.000	0.008	0.769	-0.274 [-0.280, -0.268]

Table 2: Start F1 of structure-blind baselines under the same targets, k and matching: boundary after every newline (NL), at indentation changes, and at every word start. The last column compares BLT with the strongest baseline for that language.

guages (Python 0.083/0.822; C++ 0.095/0.646; English 0.342/0.894). The asymmetry follows from the trigger rule. Since $H \leq \log B$, every boundary sits where at least $e^{\tau} \approx 3.8$ continuations were live. Identifier interiors, strings and comments are flat without being syntactic, so predictions outnumber targets 9.8 to one in Python, 6.8 in C++ and 2.6 in English, which caps precision while the same density guarantees high recall. P2 holds in the predicted direction: end margins are negative (-0.017, -0.035, -0.079), so closing delimiters are anti-aligned.

Why P3 came out reversed. The proposal predicted large BPP variance in code and near-chance alignment in prose. Mean bytes per patch (BPP) is 7.86 in C++, 5.68 in Python and 3.77 in English. The first half holds: within-file BPP variance is 527.8 in C++ and 69.4 in Python against 10.3 in English, because boilerplate stretches alternate with dense logic. The second half fails: the prose margin +0.348 exceeds the code margin +0.129. Two facts explain this. First, a benepar constituent always begins at a word, and the patcher fires at word onsets, so the word-start rule alone reaches F1 0.769 in English. Prose "alignment" is word segmentation measured against a parse. Second, F1 margins are not comparable across target densities. English has far more targets per prediction, which raises both observed and null F1. On the scale-free ratio to the null, code aligns *more* strongly than prose (4.7× Python, 4.6× C++, 3.4× English, file-bootstrap intervals in Appendix A). The reversal

Lang.	Type	Opener	Starts	R	H_0 R	ΔR
Python	`assignment`	open	39	0.872	0.156	+0.715
Python	`binary_operator`	open	7,276	0.683	0.184	+0.498
Python	`call`	open	33,643	0.766	0.168	+0.598
Python	`expression_statement`	open	43,709	0.864	0.169	+0.695
Python	`for_statement`	det.	2,829	0.934	0.162	+0.772
Python	`function_definition`	det.	485	0.891	0.166	+0.724
Python	`if_statement`	det.	10,084	0.903	0.158	+0.745
Python	`return_statement`	open	7,010	0.798	0.249	+0.550
Python	`while_statement`	det.	263	0.954	0.161	+0.794
C++	`assignment_expression`	open	8,078	0.452	0.133	+0.319
C++	`binary_expression`	open	285,064	0.506	0.144	+0.362
C++	`call_expression`	open	456,491	0.595	0.138	+0.457
C++	`declaration`	open	249,479	0.773	0.142	+0.632
C++	`expression_statement`	open	486,954	0.655	0.138	+0.517
C++	`for_statement`	det.	21,326	0.823	0.136	+0.686
C++	`function_definition`	det.	101,613	0.768	0.134	+0.635
C++	`if_statement`	det.	126,003	0.795	0.146	+0.649
C++	`return_statement`	open	93,611	0.562	0.144	+0.418
C++	`while_statement`	det.	4,102	0.942	0.150	+0.792

Table 3: Start recall by constituent type (P4). Types are rematched separately, so rows are not additive. A start shared by several nodes is credited to the outermost one, which is why nested types such as Python `assignment` have few starts. Opener class follows the empirically checked `taxonomy.json`.

therefore says that the absolute margin depends on target density, not that prose carries more structure.

Example and deliverable (4). The patcher's segmentation of a short Python function, with patch starts marked by `|`:

```
d|e|f |f|i|b|onacci|(|n|):
|if n |< |2|:
|return |n|
|return |fibonacci(n |- 1) |+
|fibonacci(n - 2)
```

Boundaries open statements after indentation, and the bytes after `if` and `return` stay inside one patch. An identifier is split at nearly every byte on first use and kept whole on reuse. Figure 1 shows the per-byte entropy heatmaps for a Python file, a C++ file and an English paragraph, with BPP and patch-length distributions per language (frozen teacher run).

Why P4 came out reversed, and what the forced bytes do. By type, forced openers have larger start-recall margins than open-ended constructs (Table 3; mean +0.759 vs. +0.611 in Python, +0.691 vs. +0.451 in C++), the opposite of P4. The mid-report's conjecture was that a start boundary sits *before* the keyword, where the choice of statement

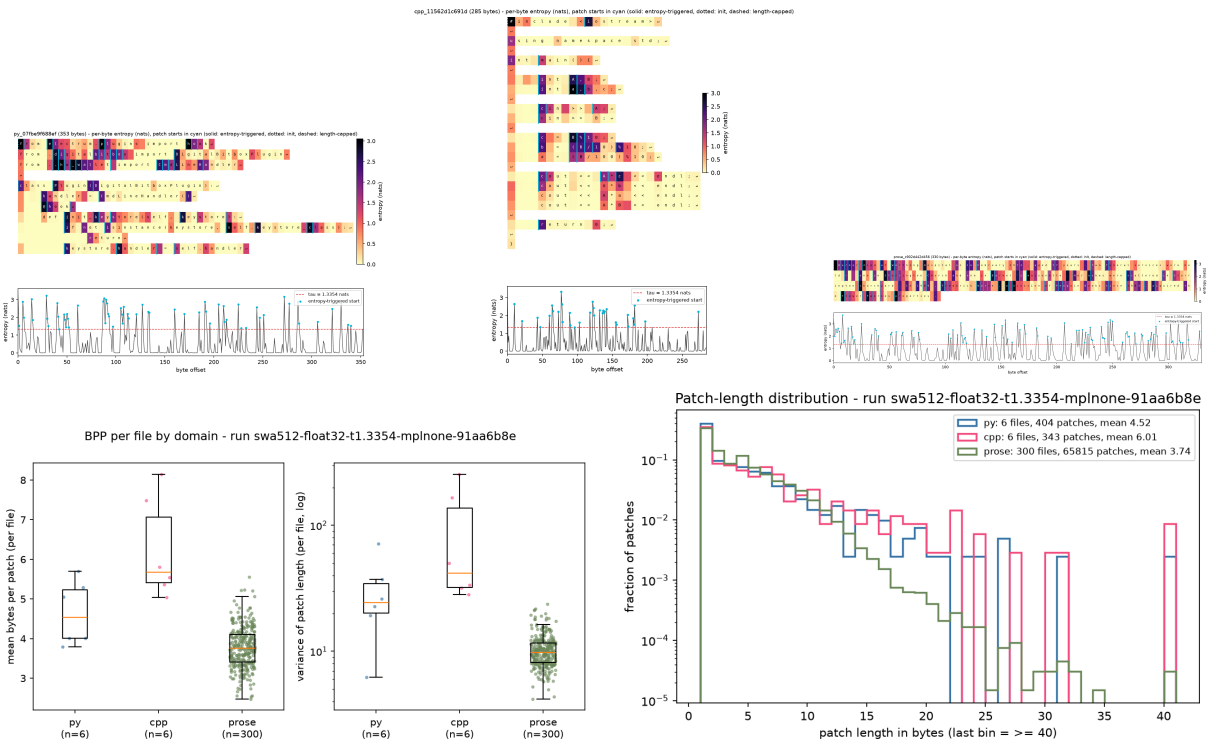


Figure 1: per-byte entropy heatmaps for a Python file, a C++ file and an English paragraph, and BPP and patch-length distributions per language (frozen teacher run).

is still open, while the forcing predicted by the proposal applies to the bytes after it. We tested this directly. For every primary target start s we take its first token (the identifier run at s: the keyword for forced openers) and score the boundary rate at fixed positions against the same 10,000 null draws as the frozen scorer, so the start rows reproduce the frozen type recalls exactly (Table 4). The rest of the keyword is segmented below chance (Python -0.130, C++ -0.088), and the byte right after it almost never starts a patch (rate 0.000 and 0.035 against null rates of 0.159 and 0.140). For open-ended constructs the byte after the first token stays near the null (-0.014, -0.001). The boundary reappears where choice reopens. In Python the next non-space byte begins the condition, and its boundary rate is 0.895 (+0.736). In C++ that byte is the equally forced `(` (-0.144 after `if`), and the boundary moves one byte further, to the condition, at offset four (rate 0.377; Figure 4). `return`, which the taxonomy classes as open-ended, behaves the same way after its keyword (-0.186, -0.139). P4's mechanism therefore holds at the bytes it is about: once a keyword is committed, its continuation is predictable and is segmented far below chance. A start-offset metric cannot see this, because the start byte is exactly where the forcing has not yet happened.

Lang.	Position	Forced opener rate	Forced opener Δ [95% CI]	Open-ended rate	Open-ended Δ [95% CI]
Python	start s	0.910	+0.751 [0.744, 0.757]	0.809	+0.633 [0.625, 0.640]
	rest of first token	0.029	-0.130 [-0.133, -0.127]	0.094	-0.065 [-0.068, -0.063]
	byte after token	0.000	-0.159 [-0.161, -0.157]	0.156	-0.014 [-0.017, -0.010]
	next non-space byte	0.895	+0.736 [0.728, 0.743]	0.178	-0.001 [-0.004, 0.002]
C++	start s	0.789	+0.649 [0.637, 0.659]	0.623	+0.483 [0.453, 0.505]
	rest of first token	0.048	-0.088 [-0.092, -0.085]	0.087	-0.050 [-0.054, -0.046]
	byte after token	0.035	-0.105 [-0.109, -0.102]	0.139	-0.001 [-0.006, 0.003]
	next non-space byte	0.161	+0.021 [0.013, 0.029]	0.241	+0.101 [0.089, 0.110]

Table 4: P4 follow-up: boundary rate at fixed positions relative to each constituent start s, against the same 10,000 null draws as the frozen scorer (Δ = rate $- H_0$ rate). The first token is the identifier run starting at s: the keyword (`if`, `for`, `while`, `def`) for forced openers.

O3: Python versus C++ on a common depth scale. Raw tree-sitter depth is not comparable across languages: C++ statements sit under more scaffolding nodes than Python ones, and benepar trees are collapsed differently again. We therefore count depth in constituents, as the number of stored target-type nodes that strictly enclose a node, and also normalise it within each file (Figure 2). On this common scale all three languages align most at the outermost constituents. From one enclosing constituent to seven, the margin falls from +0.707 to +0.519 in Python, from +0.553 to +0.387 in C++, and from +0.714 to +0.601 in English. Python exceeds C++ at every depth both languages share and

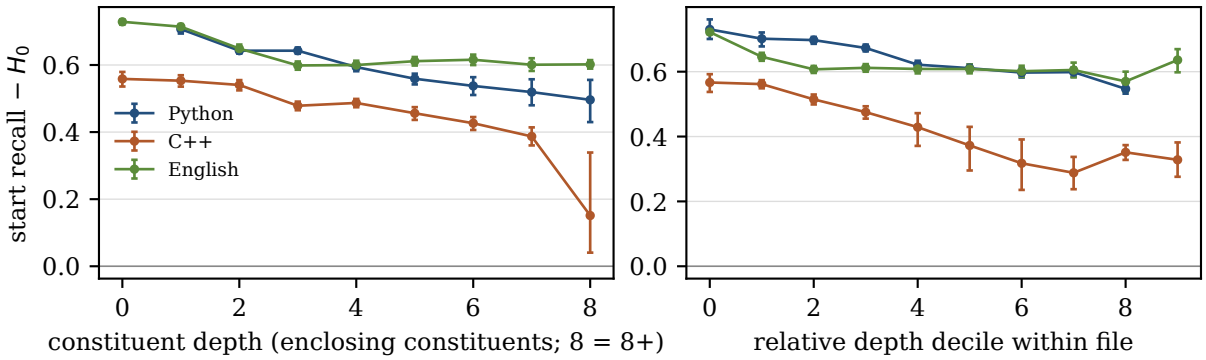


Figure 2: O3: held-out start recall minus mean H_0 recall by scaffold-free depth. Left: the number of enclosing constituents (raw tree-sitter depth also counts grammar scaffolding such as `block`, `compound_statement` and `argument_list`, whose amount differs by language). Right: within-file relative depth decile. Strata with at least 200 starts; 10,000 frozen null draws; file-bootstrap 95% intervals.

at every relative-depth decile. English is nearly flat below the top levels, consistent with word-onset alignment holding at every level of a parse. The indentation-versus-brackets contrast that O3 asked about shows up in the baselines rather than in the depth profile. In Python the indentation rule is the strongest baseline, because indentation *is* the grammar. In C++ word starts are the strongest baseline, and BLT's deficit to them (-0.173) is smaller than its deficit to indentation in Python (-0.242).

O4: robustness to noise, by rate. At the frozen rates (20% of lines or keywords, 50% of identifiers) every noised margin stays at or above +0.099 (Python) and +0.118 (C++), and the largest drop is -0.016 for indentation noise in Python (Appendix Table 13). The four perturbations are those the proposal named: missing semicolons (C++ only), localised indentation changes, camelCase↔snake_case inversion of identifiers, and typos in syntax keywords. To test whether this "graceful degradation" depends on the rate, we rebuilt the same 1,000 files per language with each noise type at 5, 10, 20 and 40% (case inversion at 25, 50 and 100%), and scored them with the unchanged O4 scorer (Figure 3). Only indentation noise does damage that grows steadily with rate, roughly in proportion to the share of lines touched: from -0.005 at 5% to -0.029 at 40% in Python, and from -0.004 to -0.023 in C++. This is the expected signature of a patcher whose statement boundaries ride on line layout. Keyword typos barely move alignment (-0.007 and -0.002 at 40%). They do lower bytes per patch in proportion to rate (-0.43 and -0.60 at 40%), because a misspelt keyword is unpredictable and is split into more patches. Renaming identifiers costs at most -0.001 (Python) and -0.002 (C++), with no trend between 25% and 100% of names, consistent with identifiers already being near-arbitrary for the patcher. Every noised margin stays at or above +0.086.

O5: is there a semantic residual near memory-unsafe code? We first audited the region labels (`docs/O5_LABEL_AUDIT.md`). Re-running the tree-sitter queries reproduced 12,609/12,609 frozen regions exactly, and an independent lexical labeller agreed on every label for 433/435 audited statements. The two disagreements were address-of operations inside casts, which tree-sitter labels correctly. Following the proposal's compute-profiling metric we model two outcomes, boundary density and inverse-BPP compute density. The frozen regression gives unsafe statements +0.0069 [0.0043, 0.0095] extra boundaries per byte after file fixed effects and identifier controls. Refitting it as a nested sequence (our refit reproduces the frozen coefficients) changes the reading (Table 5). With file fixed effects only, unsafe statements receive *fewer* boundaries (-0.0083 [-0.0127, -0.0039]), because they use more predictable identifiers. The positive coefficient appears only after conditioning on identifier entropy, and identifier share alone does not flip it. The conditional effect is 4.2% of the comparison statements' mean density, and 7% of the effect of one standard deviation of identifier entropy. It is most plausibly carried by the unsafe operator bytes themselves (`*`, `&`, `new`), which are surface tokens. O5 therefore finds no evidence that the patcher allocates extra compute to memory risk. What survives the confound is a small conditional surface effect, and the unconditional effect points

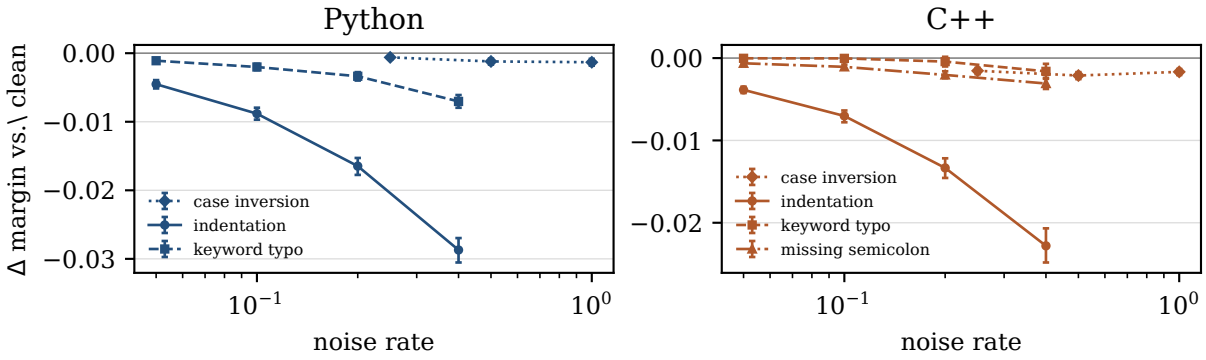


Figure 3: O4 rate sweep: change in the start margin over H_0 relative to the clean files, by noise type and rate (log scale), on the same held-out files. Targets are clean constituents mapped to noised offsets; 1,000 permutations per file; paired file-bootstrap 95% intervals.

Model	Boundary density	Inverse BPP
file FE only	-0.0083 [-0.0127, -0.0039]	-0.0024 [-0.0066, 0.0017]
+ statement type	-0.0078 [-0.0122, -0.0035]	-0.0021 [-0.0062, 0.0020]
+ identifier entropy	+0.0063 [0.0037, 0.0089]	+0.0109 [0.0084, 0.0133]
+ identifier share	-0.0080 [-0.0123, -0.0036]	-0.0022 [-0.0063, 0.0019]
full (frozen)	+0.0069 [0.0043, 0.0095]	+0.0112 [0.0087, 0.0137]

Table 5: O5: the memory-unsafe coefficient [95% CI] as controls are added (OLS, file fixed effects, file-clustered CR1 intervals; 12,609 statements in 1,500 clean-parse C++ test files, 5,271 unsafe). The last row is the frozen model.

the other way.

Are the intervals too narrow? The 8,000 Python functions come from only 315 repositories. Resampling repositories instead of files widens the Python interval 3.7× to [0.112, 0.127], which still excludes zero. Tree-sitter error recovery affects 28.0% of C++ test files, and their margin differs from clean parses by +0.004 [-0.005, 0.015]. Files longer than 8,191 bytes, whose context resets at chunk edges, make up 31.5% of C++ test files but 86% of its bytes, and their margin differs by -0.002 [-0.008, 0.004]. Every subset's margin is recomputed exactly from the frozen per-file counts (Appendix Table 11). Under $k = 1$ and $k = 2$ as test-split sensitivity checks the margins shrink, as the calibration curve predicts (Appendix Table 12), and BLT never overtakes the strongest baseline. Scoring HF's native full-attention mode instead, which is not faithful beyond 512 tokens, cuts the margin to +0.023 (Python), +0.008 (C++ files under 8,192 bytes) and +0.150 (English test files), against +0.120, +0.131 and +0.348 with the restored window on the same files, so every result here depends on that restoration.

5 Beyond the Proposal: a Distilled Patcher

The proposal needed no training. We added one experiment, labelled here as an extension and separately pre-registered: can a much smaller byte model reproduce the patcher's boundaries, and does structural alignment survive distillation? A 14.3M-parameter student (384 dimensions, 8 layers, 6 heads, BLT-style byte transformer) was trained from scratch by online distillation, $\alpha\,\mathrm{CE} + (1-\alpha)T^2\,\mathrm{KL}(p_T \| p_S)$ with $\alpha = 0.1$ and $T = 2$, on ≤512-byte windows of a disjoint extension corpus. That corpus has 29,000 files: CodeSearchNet functions from repositories not in the evaluation set, WikiText paragraphs from unused articles, and Stack C++ from other repositories, split by repository or article. On held-out extension windows the student's boundaries reach F1 0.871 against the teacher's. Dropping the teacher ($\alpha = 1$, same data, seed and schedule) reaches 0.789, and placing a boundary at every word start reaches 0.475 (Appendix Table 17). The $\alpha = 1$ student predicts the true next byte better (1.17 vs. 1.24 bits/byte; teacher 1.14), so distillation trades a little compression for agreement with the teacher's segmentation.

Held-out alignment. We extracted the student on all 16,300 corpus files with stride-1 fresh 512-byte windows, the regime it was trained in. Before that, the extractor had to reproduce the committed golden-fixture scores on the cluster GPU, and it did so exactly (largest F1 deviation 0.0000). The student then went through the frozen C-v1 pipeline with its own calibration record, which again selected $k = 0$ everywhere (Table 6). It keeps 91%, 92% and 87% of the teacher's margin (Python, C++, English). The paired deficits are small but re-

solved (-0.011 [-0.011, -0.010] in Python), and the student, like the teacher, stays below the structure-blind baselines. The student fires more often (mean BPP 5.17 vs. 5.68 in Python), which buys recall at the cost of precision. A control on 300 test files per code language restricts the *teacher* to the student's fresh 512-byte windows. This costs +0.004 [0.003, 0.005] (Python) and +0.004 [0.003, 0.005] (C++) of F1, about 38% and 43% of the student's deficit on the same files (Appendix Table 15). The 512-byte context explains part of the gap, and model capacity the rest.

Threshold and tolerance. The checkpoint's τ was set for compression, not alignment. We swept $\tau \in \{0.8, 1.0, 1.335, 1.7, 2.0, 2.5\}$ on calibration files only and pre-registered τ^* as the argmax of the median per-file margin, with ties going to the smaller τ. Two readings of the criterion (per-file and pooled null means) agreed. For the teacher $\tau^* = 2$ in all three languages (calibration gain +0.037 in Python), and for the student $\tau^* = 2$ (Python), 2 (C++) and 2.5 (English) (Appendix Figure 5). The student's English optimum sits at the edge of the grid, where its entropy runs higher than the teacher's. The pre-registered grid was not extended after seeing this, so each language with its own τ^* was confirmed in a separate run. After re-freezing k under τ^*, each model's confirmatory test run was scored exactly once (Table 7): the teacher's margin rises to +0.153 (Python), +0.158 (C++) and +0.426 (English), a paired gain of +0.033 [0.032, 0.034] in Python, and it remains below the strongest baseline (-0.210, -0.148, -0.215). The student's margin at its own τ^* is +0.147, +0.156 and +0.460, also below the baselines. A later test-file run of the whole grid, which selects nothing, peaks at the same τ^* (Figure 5). Fewer, higher-entropy boundaries are better aligned, but no threshold turns entropy into a parser.

The confirmatory English test. The 300 evaluation paragraphs were inspected during exploratory analysis before the protocol was frozen. We therefore pre-registered a holdout of the 120 lowest-ranked WikiText articles that neither evaluation nor distillation used, 4,107 paragraphs with benepar structure, and scored it once after team sign-off (Table 8). The teacher's margin is +0.350, against +0.348 on the evaluation paragraphs, and the student's is +0.305. Both stay below word starts (-0.273 and -0.311), so the English findings replicate on untouched text.

Cost. On one RTX 2080 Ti the student's forward pass is 5.0× faster than the teacher's at the same input (Appendix Table 16), and 4.8× faster under the rolling extractor used for both student runs. The native teacher processes a file in a single sliding-window pass at 13,246 tokens/s. The student architecture does not implement that pass, which is why its full-corpus extraction took about 20 GPU-hours per shard. A deployment-grade student would need the same single-pass kernel.

Byte balancing and domain adaptation. Two follow-ups ran after the main analysis, on the pre-registered 600-file code subset of Table 15 plus the English test split (Appendix Table 9). Full-corpus extraction of these models did not fit before the deadline, so both are exploratory. C++ is 73% of the distillation bytes. A student trained with every domain drawn to an equal expected byte share, and otherwise identical, agrees with the teacher's boundaries on held-out windows at F1 0.860 (distilled student: 0.871), and its margin differs from the distilled student's by -0.002 [-0.003, -0.001] (Python), -0.002 [-0.0035, -0.0001] (C++) and +0.010 [0.008, 0.012] (English). The pre-registered adaptation arm initialises the teacher architecture from the patcher and trains it with the same loss on the same data, then reads files through the same fresh 512-byte windows as the student. Its margin differs from the student's by +0.006 [0.004, 0.008], +0.005 [0.003, 0.007] and +0.029 [0.026, 0.032]. The pre-registered criterion, a gain larger than the width of the student's confidence interval in at least two domains, is not met (1 of 3). Balancing trades code for prose: it raises English window agreement (0.853 against 0.834) and the English margin, and lowers both in code. In code the adapted teacher's margin matches the unadapted teacher read through the same windows (Table 9), so fine-tuning recovers the teacher rather than improving on it. Its gain over the student is the capacity the student gave up, and what separates both from the native teacher is the 512-byte window.

6 Verdict on H1 and Deliverables

The proposal defined support for H1 as: P1 clears both baselines with non-overlapping intervals, P2 holds in the predicted direction, P3 shows a code/prose dissociation, and P4's stratification appears. Judged that way, H1 is only partly supported. P1 clears the density-matched null (H0) in every language but not the whitespace and word-

	Teacher (BLT patcher, 99.5M)				Student (distilled, 14.3M)					
Lang.	P	R	F1	Δ	P	R	F1	Δ	Student−teacher Δ [95% CI]	vs. best base.
Python	0.083	0.822	0.152	+0.120	0.076	0.859	0.140	+0.109	-0.011 [-0.011, -0.010]	-0.253
C++	0.095	0.646	0.165	+0.129	0.087	0.732	0.155	+0.119	-0.011 [-0.012, -0.009]	-0.183
English	0.342	0.894	0.495	+0.348	0.301	0.965	0.459	+0.304	-0.044 [-0.048, -0.040]	-0.310

Table 6: The distilled student on the frozen held-out split, scored by the frozen C-v1 pipeline with its own calibration record (which also selected $k = 0$ in every language). Δ is F1 minus the mean H_0 F1 (10,000 fixed-EOF permutations); the student−teacher interval is a paired file bootstrap; the last column is the student's F1 minus the strongest structure-blind baseline (indentation for Python, word-start otherwise).

Model	Lang.	τ^*	k^*	F1 at τ=1.335	F1 at τ^*	Δ at τ^* [95% CI]	vs. best baseline [95% CI]
Teacher	Python	2	0	0.152	0.184	+0.153 [0.150, 0.155]	-0.210 [-0.215, -0.205]
Teacher	C++	2	0	0.165	0.190	+0.158 [0.153, 0.163]	-0.148 [-0.156, -0.142]
Teacher	English	2	0	0.495	0.554	+0.426 [0.419, 0.433]	-0.215 [-0.221, -0.210]
Student	Python	2	0	0.140	0.178	+0.147 [0.144, 0.150]	-0.216 [-0.221, -0.211]
Student	C++	2	0	0.155	0.190	+0.156 [0.151, 0.160]	-0.149 [-0.155, -0.143]
Student	English	2.5	0	0.459	0.583	+0.460 [0.453, 0.468]	-0.186 [-0.193, -0.180]

Table 7: The single confirmatory test-split run at the pre-registered τ^* (argmax over $\{0.8, 1.0, 1.335, 1.7, 2.0, 2.5\}$ of the median per-file margin on calibration files only, ties to the smaller τ), with k re-frozen on the calibration split under τ^* by the frozen criterion. This run was scored once per model.

Model	P	R	F1 [95% CI]	H_0	Δ	p	vs. word-start [95% CI]
Teacher	0.346	0.901	0.500 [0.498, 0.502]	0.150	+0.350	≤1e-4	-0.273 [-0.275, -0.272]
Student	0.304	0.966	0.463 [0.461, 0.464]	0.158	+0.305	≤1e-4	-0.311 [-0.312, -0.310]

Table 8: The confirmatory English result: the pre-registered holdout of 4,107 paragraphs from 120 WikiText-103 articles unused by evaluation and distillation, scored once at the frozen $k = 0$ (10,000 permutations). Word-start F1 = 0.774.

start baselines. P2 holds. P3 is reversed in absolute margin, although the ratio to the null separates code from prose. P4 is reversed at the start byte, yet its mechanism holds at the bytes after the keyword. Each failure is reported with the test that explains it. The proposal's five deliverables are all delivered: (1) the boundary-extraction pipeline (`stream_a`), producing per-byte entropies, tagged boundaries and BPP statistics; (2) the tree-sitter and benepar mapping engine with the memory-unsafe queries (`stream_b`); (3) the evaluation harness (`escape_scoring`, `escape_eval`) with P1–P4, both baselines, the permutation test and depth and node-type stratification; (4) the entropy heatmaps and BPP profiles (Figure 1); and (5) this report and the presentation, covering O1–O5 including the negative results. The code, per-file results, tests and the build that regenerates every number in this report ship in `Trimax-Final.zip`.

7 Conclusion

BLT's entropy boundaries sit at constituent starts three to five times more often than a null with the same patch lengths, in all three languages, on held-out files, in a pre-registered English holdout, and in a 7× smaller distilled student. Yet the burden we set in the proposal is not met: indentation and word-start rules segment better, and no threshold changes that. The three failed predictions fail for interpretable reasons that the follow-up tests isolate. Margins scale with target density (P3). Forcing acts on the bytes after an opener, not on its start (P4). The memory-unsafe "residual" is a conditional surface effect whose unconditional sign is negative (O5). Entropy-based patching tracks where uncertainty is high: at the opening of statements, at word onsets, and at fresh identifiers. Syntax shapes that signal without being what it measures.

Ethical Considerations

We analyse existing text and code without executing corpus programs. Dataset provenance and licences accompany restored inputs; CodeSearchNet carries no per-file licence.

Limitations and Future Work

Limitations. The checkpoint is a third-party conversion with a sliding window we restored; bitwise identity with the gated original was not tested, and its patch-length distribution was not compared with

statistics published for the original model, as the proposal planned. English targets are predicted benepar parses, and the 300 evaluation paragraphs were seen during exploration, so only the holdout is confirmatory for English. The O5 labels are syntactic: no types, aliasing or bounds are known, and O5 is restricted to clean-parse C++. The student was evaluated only with the rolling extractor and at one size. Its $\alpha = 1$ control and the threshold sweep use the same corpus and seed, so between-seed variance is not measured. The byte-balanced student and the adapted teacher were scored only on the 600-file code subset and the English test split. We did not evaluate a subword-tokenizer baseline, so O4 establishes robustness only for the tested perturbations and not relative to subword tokenizers.

Future work. A parser-supervised teacher, planned as a further extension, was not run. Open items are a subword-tokenizer baseline for O4, multiple training seeds for the student, full-corpus scoring of the byte-balanced and adapted models, a single-pass student kernel for deployment, and a bitwise comparison with the gated original checkpoint.

References

Michael R. Brent. 1999. An efficient, probabilistically sound algorithm for segmentation and word discovery. *Machine Learning*, 34:71–105.

Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. 2022. CANINE: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 10:73–91.

Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547, Online. Association for Computational Linguistics.

Sharon Goldwater, Thomas L. Griffiths, and Mark Johnson. 2009. A bayesian framework for word segmentation: Exploring the effects of context. *Cognition*, 112(1):21–54.

John Hewitt and Christopher D. Manning. 2019. A structural probe for finding syntax in word representations. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4129–4138, Minneapolis, Minnesota. Association for Computational Linguistics.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.

Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436*.

Nikita Kitaev and Dan Klein. 2018. Constituency parsing with a self-attentive encoder. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2676–2686, Melbourne, Australia. Association for Computational Linguistics.

Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro von Werra, and Harm de Vries. 2022. The stack: 3 TB of permissively licensed source code. *arXiv preprint arXiv:2211.15533*.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer sentinel mixture models. In *International Conference on Learning Representations*.

Artidoro Pagnoni, Ramakanth Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason E Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman, and Srini Iyer. 2025. Byte latent transformer: Patches scale better than tokens. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9238–9258, Vienna, Austria. Association for Computational Linguistics.

Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany. Association for Computational Linguistics.

Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. 2022. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306.

A Supplementary Tables

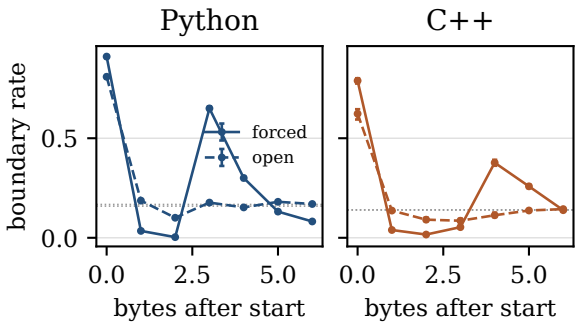


Figure 4: Boundary rate at each byte offset after a constituent start, forced openers (solid) vs. open-ended (dashed); dotted line: null rate.

Lang.	Files	Teacher (native) F1	Δ	Teacher, 512 windows F1	Δ	Student (distilled) F1	Δ	Student, byte-balanced F1	Δ	Teacher, adapted (512 windows) F1	Δ
Python	300	0.143	+0.114	0.139	+0.110	0.132	+0.103	0.130	+0.101	0.139	+0.109
C++	300	0.162	+0.125	0.158	+0.121	0.152	+0.115	0.150	+0.114	0.158	+0.121
English	240	0.495	+0.348	–	–	0.459	+0.304	0.467	+0.314	0.482	+0.333

Table 9: Follow-up arms on the pre-registered subset (300 test files per code language, 600–2,000 bytes, as in Table 15) and the English test split, $k = 0$, 10,000 permutations. All models except the native teacher see fresh 512-byte windows. The teacher-512 control was not extracted for English.

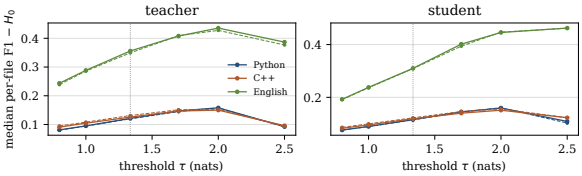


Figure 5: Threshold sweep: median per-file F1 margin over H_0 ($k = 0$). Solid: the pre-registered sweep on calibration files (10,000 permutations) that fixed τ^*. Dashed: the same grid on test files, scored only after both confirmatory runs as an exploratory check (1,000 permutations); its maximum is the calibration τ^* in 3/3 languages for the teacher and 3/3 for the student, and every grid point trails the strongest baseline by at least 0.148 F1. Dotted: the checkpoint's $\tau = 1.335$.

Lang.	F1	H_0 F1	Δ	F1 / H_0 [95% CI]
Python	0.152	0.032	+0.120	4.73 [4.69, 4.78]
C++	0.165	0.036	+0.129	4.61 [4.39, 4.76]
English	0.495	0.147	+0.348	3.36 [3.31, 3.42]

Table 10: P3 on a scale-free axis. The difference Δ grows with target density, the ratio does not: code aligns more strongly than prose relative to its own null. Frozen held-out run; paired file bootstrap.

Model	Subset (files)	Δ F1 vs. H_0 [95% CI]
Teacher	C++ clean parse (4,611)	+0.127 [0.117, 0.135]
	C++ error recovery (1,789)	+0.131 [0.126, 0.136]
	C++ >8,191 bytes (2,016)	+0.129 [0.124, 0.134]
	C++ ≤8,191 bytes (4,384)	+0.131 [0.129, 0.133]
	Python, file bootstrap	+0.120 [0.118, 0.122]
	Python, repo-cluster bootstrap (315 repos)	+0.120 [0.112, 0.127]
Student	C++ clean parse (4,611)	+0.120 [0.112, 0.125]
	C++ error recovery (1,789)	+0.118 [0.114, 0.123]
	C++ >8,191 bytes (2,016)	+0.118 [0.114, 0.122]
	C++ ≤8,191 bytes (4,384)	+0.121 [0.119, 0.123]
	Python, file bootstrap	+0.109 [0.107, 0.111]
	Python, repo-cluster bootstrap (315 repos)	+0.109 [0.101, 0.117]

Table 11: Held-out start margin on subsets, recomputed exactly from the frozen per-file counts (the null mean is linear in each file's expected matches). File-level 95% bootstrap intervals except where stated.

	Calibration: median per-file Δ					Test: pooled Δ		
Lang.	k=0	1	2	4	8	0	1	2
Python	+0.121	+0.084	+0.056	+0.029	+0.010	+0.120	+0.086	+0.060
C++	+0.126	+0.088	+0.061	+0.033	+0.014	+0.129	+0.090	+0.063
English	+0.357	+0.179	+0.077	+0.018	+0.002	+0.348	+0.177	+0.080

Table 12: Tolerance sweep for the teacher. Calibration columns are the frozen record's selection curve (the criterion picks $k = 0$ everywhere); test columns at $k = 1, 2$ are sensitivity only (1,000 permutations) and select nothing.

Lang.	Noise	Edits	ΔF1 vs. H_0	Change vs. clean [95% CI]	BPP	Change vs. clean [95% CI]
Python	clean	0	+0.115	–	5.61	–
Python	camel↔snake	4,601	+0.114	-0.001 [-0.002, -0.001]	5.56	-0.05 [-0.06, -0.04]
Python	indentation	4,416	+0.099	-0.016 [-0.018, -0.015]	5.61	+0.00 [-0.00, 0.01]
Python	keyword typo	1,626	+0.112	-0.003 [-0.004, -0.003]	5.38	-0.23 [-0.25, -0.21]
C++	clean	0	+0.131	–	7.88	–
C++	camel↔snake	83,037	+0.129	-0.002 [-0.003, -0.002]	7.78	-0.10 [-0.11, -0.09]
C++	indentation	68,746	+0.118	-0.013 [-0.015, -0.012]	7.75	-0.13 [-0.14, -0.12]
C++	keyword typo	17,884	+0.131	-0.000 [-0.001, 0.000]	7.54	-0.34 [-0.36, -0.31]
C++	missing ;	25,972	+0.129	-0.002 [-0.002, -0.002]	7.82	-0.06 [-0.06, -0.05]

Table 13: Objective 4: robustness to code noise on the same held-out files per language. Each noise type is applied alone; targets are the clean constituents mapped to noised offsets; k is the frozen value. The null uses 1,000 fixed-EOF permutations per file; intervals are paired file bootstraps of noised minus clean.

Lang.	Files	BLT	H_0	Line	Word	BLT−H_0 [95% CI]
Python	6,400	0.351	0.342	0.546	0.342	+0.008 [0.007, 0.010]
C++	6,188	0.372	0.356	0.606	0.409	+0.016 [0.013, 0.018]
English	240	0.377	0.334	0.049	0.397	+0.042 [0.040, 0.045]

Table 14: Secondary metric: mean best single-patch IoU over unique constituent spans, held-out split (files with at least one span). H_0 averages 100 fixed-EOF permutations; Line and Word are newline and word-start segmentations.

		Native		Teacher-512		Student		
Lang.	Files	F1	Δ	F1	Δ	F1	Δ	Native−512 F1 [95% CI]
Python	300	0.143	+0.114	0.139	+0.110	0.132	+0.103	+0.004 [0.003, 0.005]
C++	300	0.162	+0.125	0.158	+0.121	0.152	+0.115	+0.004 [0.003, 0.005]

Table 15: Context control on the pre-registered subset (300 test files per code language, 600–2,000 bytes): the frozen teacher (native 512-byte sliding window), the same teacher restricted to the student's fresh 512-byte windows, and the student. $k = 0$, 10,000 permutations.

Model	Params	Forward, 32×513 tok/s	MiB	Rolling extraction bytes/s	MiB
Teacher	99.5M	44,356	1,219	106	1,235
Student	14.3M	222,549	714	506	1,011
Teacher, native single pass		13,246	1,364	–	–

Table 16: Inference cost on one RTX 2080 Ti, float32. Rolling extraction (one fresh 512-byte window per byte) is how both student runs were extracted; the native teacher processes a file in one sliding-window pass, which the student architecture does not implement.

Student	Boundary F1	r	KL	bits/byte
Distilled (α=0.1)	0.871	0.946	0.135	1.24
No teacher (α=1)	0.789	0.864	0.312	1.17
Untrained	0.386	-0.037	4.821	8.05
Word starts (no model)	0.475	–	–	–

Table 17: What distillation adds: agreement with the teacher on held-out extension-corpus windows (F1 of the student's boundaries against the teacher's at $\tau = 1.335$; Pearson r and KL of per-byte entropy). Same data, architecture, seed and schedule; only the loss differs. Teacher bits/byte: 1.14.